

Consumer Safety Research Institute

Data Preparation and Modeling Report

Table of Contents

Introduction.....	4
1. Data Preparation	6
1.1 Data Selection Rationale.....	6
1.1.1 Variable Selection - A Priori Logical Selection (Phase 1)	6
1.1.2 Temporal Selection	9
1.1.3 Record Selection	9
1.1.4 Sampling Strategy	9
1.1.5 Connection to Data Science Goals	11
1.2 Data Cleaning Process.....	13
1.2.1 Missing Data Treatment.....	13
1.2.2 Outlier Detection and Handling.....	15
1.2.3 Inconsistency Resolution	16
1.2.4 Error Correction	21
1.2.5 Impact Assessment	21
1.3 Feature Engineering.....	22
1.3.1 Derived Features.....	22
1.3.2 Transformations	24
1.3.3 Temporal Features.....	25
1.3.4 Domain Specific Features	25
1.4 Data Integration and Final Preparation	30
1.4.1 Integration Approach	30
1.4.2 Data Formatting	30
1.4.3 Train/Validation/Test Splitting.....	31
1.4.4 Statistical Variable Selection (Phase 2)	31
1.4.5 Final Dataset Characteristics	32
1.4.6 Preparation Pipeline.....	37
2. Modelling.....	38
2.1 Modelling Strategy.....	38
2.1.1 Problem Framing	38
2.1.2 Algorithm Selection Rationale.....	38
2.1.3 Evaluation Strategy	38
2.1.4 Computational Considerations.....	39
2.1.5 Connection to Data Science Goals and Success Criteria	39
2.2 Model Development	41
2.2.1 Baseline Model	41
2.2.2 Primary Models	43
2.2.3 Computational Requirements	53
2.2.4 Model Interpretation	55
2.3 Model Evaluation and Comparison	58
2.3.1 Performance Metrics	58
2.3.2 Comparative Analysis	58
2.3.3 Diagnostic Evaluation	59

2.3.4	Robustness Testing.....	62
2.3.5	Organizational Metric Translation	63
2.4	Model Selection and Recommendations.....	64
2.4.1	Selected Model.....	64
2.4.2	Selection Justification	64
2.4.3	Limitations and Risks	65
Conclusion		67
References		68

Introduction

The Consumer Safety Research Institute (CSRI) is a specialized fictional nonprofit organization established in 2012 to perform independent research on consumer product safety and publish reports and insights that defines public disclosure, and aids in policy decisions. The domain of consumer product safety is currently characterized by a significant disconnect between available data and public understanding. the prevailing public discourse frequently relies on anecdotal evidence, high-profile media incidents, or biased manufacturer marketing rather than systematic, empirical analysis. This reliance on incomplete information leaves consumers, policymakers, and safety advocates without the evidence-based insights necessary for informed decision-making. This information gap creates a critical challenge in an environment where stakeholders increasingly demand rigorous, independent verification to distinguish between isolated accidents and genuine systemic risks. There is a pressing market need for an authoritative intermediary capable of translating dense surveillance data into clear, actionable intelligence.

The primary objective of the project is to develop predictive models to identify which consumer product injuries are most likely to result in hospitalization and determine what factors—product categories, demographics, injury characteristics—predict severe medical outcomes among emergency department cases, producing evidence-based findings that inform CSRI's 2025 Product Safety Spotlight report and contribute to public understanding of consumer product safety priorities. The secondary objective of the project is to Identify temporal and demographic trends in consumer product injury severity patterns, including seasonal variations, changes over time, and emerging patterns that may indicate shifting safety concerns, generate insights about demographic patterns in severe product injuries that inform targeted safety messaging, and create analytical approaches that enhance CSRI's reputation as a leading source of independent consumer safety research.

Primary data science goal is to establish statistically significant correlations between factors contributing to physical injuries and hospitalization due to the use of consumer products with a high degree of confidence; secondary goal is to create interpretable models that quantify effect sizes for public communication. This dataset description document will explain in detail the datasets used for the modelling and any major analysis work of the project.

1. Data Preparation

The data preparation phase is divided into five key stages. The subsequent sections will extensively describe our methodology for executing each:

1. Data Selection
2. Data Cleansing
3. Data Construction
4. Data Integration
5. Data Formatting

1.1 Data Selection Rationale

1.1.1 Variable Selection - A Priori Logical Selection (Phase 1)

Variable selection was executed in two iterative passes to align with CRISP-DM principles while accommodating engineered features.

In Phase-1, we perform the dimensionality reduction based on Data Quality audit and business logic. We will perform another phase of variable selection after the Feature Engineering/Data Construction phase. The NEISS dataset has a total of 26 features /variables and below is the list of features that we are planning to consider/omit.

The below table summarizes the percentage of NULL data from the consolidated dataset.

Column_Name	Count
Other_Race	6821041
Hispanic	5283928
Other_Diagnosis	6329255
Body_Part_2	6866693
Diagnosis_2	6866693
Other_Diagnosis_2	7214095
Alcohol	5283928
Drug	5283928

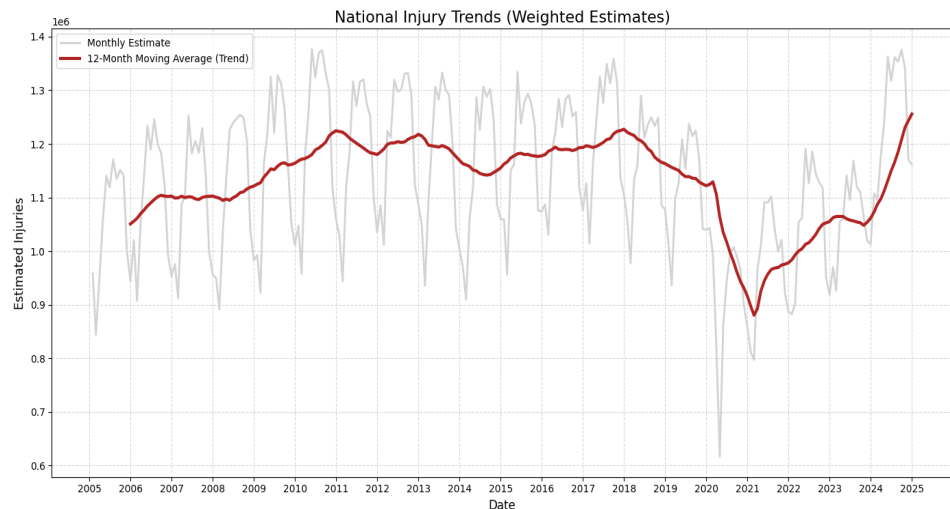
Based on the above information, the below table defines the if we need to include/exclude a feature and the corresponding reasoning.

Excel/Tab Variable	Include/Exclude	Include/Exclude Reason
CPSC_Case_Number	Exclude	This is a record identifier and does not hold any predictive value.
Treatment_Date	Include	Helps to define the treatment year/month. Valuable for timeseries analysis.
Age	Include	Can be used for prediction.
Sex	Include	Can be used for prediction.
Race	Include	Understand the demography of ER admissions.
Other_Race	Exclude	Only used when Race = 3 (Other) and 93% of the values are null.
Hispanic	Exclude	Available only from 2019 and 73% of the records have a NULL value.
Body_Part	Include	Will help to understand the relation between hospitalization and the injured body part.
Body_Part_2	Exclude	94% of the records have a NULL value. Added to log the injury to the secondary body part.
Diagnosis	Include	Will help to understand the relation between hospitalization and the injury.
Diagnosis_2	Exclude	93% of the records have a NULL value. Available only from 2019.
Other_Diagnosis	Exclude	Only used when the diagnosis is 71 - Other. 86% of the records have a NULL value.

Other_Diagnosis_2		98% of the records have a NULL value. Only used with diagnosis code 71 - Other.
Disposition	Include	This can be used to generate the target variable. (Hospitalized - Yes/No). But we exclude this feature for modelling as this variable is not available at the time of prediction.
Location	Include	Can be used to gain insights into the where the injury happened.
Fire_Involvement	Include	Use this along with the narrative field to decide if there was a fire involvement.
Alcohol	Include	Use this along with the narrative field to decide if there was an alcohol involvement.
Drug	Include	Use this along with the narrative field to decide if there was an alcohol involvement.
Product_1	Include	Helps to understand what product caused the injury.
Product_2	Include	Helps to understand what product caused the injury.
Product_3	Exclude	Only available from 2019.
Narrative	Include	Can derive insights into the nature of injury and cause.
Stratum	Exclude	Defines the hospital where the treatment happened and does not technically define the disposition.
PSU	Exclude	Defines the hospital where the treatment happened and does not technically define the disposition.
Weight	Include	Included as we can calculate national estimates using this attribute.

1.1.2 Temporal Selection

The NEISS data is available for the last 20 years. An a priori data selection was applied on the dataset to ensure longitudinal continuity and to maintain seasonal bias. The analysis used the data available 2009 – 2024.



We used stratified sample by year and the target variable to simulate the data sampled in all the 3 data sets. More details of the sampling can be found in the section 1.1.4.

1.1.3 Record Selection

The record selection dictates which rows from the initial dataset makes to the final dataset. No specific record selection was performed other than temporal selection.

1.1.4 Sampling Strategy

Use Stratified Sampling:

To construct the Train (70%), Validation (15%), and Hold-Out (15%) subsets, a temporally stratified sampling technique is utilized.

To address the dual challenges of a highly imbalanced target variable and temporal data anomalies, a composite stratification methodology was engineered for the train-test split. A synthetic composite key was generated by concatenating the 'Treatment_Year' and the binary 'Disposition' (Hospitalized vs. Released) variables. By utilizing this composite key as the stratification target, the partitioning algorithm mathematically guaranteed that the historical temporal distribution—including macro-anomalies like the 2020 pandemic—and the severe injury class imbalance were perfectly preserved across all data folds. This approach ensures the predictive model is evaluated on a truly representative cross-section of the entire historical timeline without geographic or temporal clustering.

Given the significant shifts in consumer behavior and injury mechanisms resulting from macro-environmental factors (e.g., the 2020 COVID-19 pandemic), a standard randomized split posed a risk of temporal bias. By stratifying the split explicitly on a special composite variable, the exact annual distribution of the overarching dataset was perfectly preserved across all three data partitions, ensuring the machine learning algorithms could identify consistent longitudinal trends without being skewed by isolated annual anomalies.

The stratified data partitioning was accomplished by creating a composite key composed of Data Year and Hospitalization (Yes or No – derived from Disposition Code). By stratifying your train/test split on the new data_year_hospitalized key, helps guaranty two things simultaneously. Each year is represented equally in the training and test sets and the exact ratio of severe vs. minor injuries is preserved

within every single year. This perfectly insulates the machine learning model against historical anomalies (like COVID-19 lockdowns in 2020) while maintaining the real-world imbalance of your target variable.

The disposition feature is highly imbalanced and more than 85% of the cases are “treated and released” thereby creating a massive imbalance. If we train the model with this imbalance, the model will always classify the injury as minor (Majority class). We need to hence apply Majority Under sampling on the training set there by forcing the model to learn the mathematical difference between a minor Vs major injury. This step needs to be performed when splitting the data into train, evaluation and hold out sets.

Balancing the training data:

To address heavy class imbalance, Random Under sampling of majority class is applied on the training dataset to establish a 1:1 class ratio that optimizes, algorithmic learning without distorting the data distribution in the hold-out/test dataset. This step will need to be performed during the model training phase.

1.1.5 Connection to Data Science Goals

Sourcing Predictive Signals

To achieve the goal of building a highly accurate triage model, the selected raw dataset must contain variables capable of exposing complex, real-world patterns. By deliberately including patient demographics, incident locales, specific consumer product codes, and raw triage narratives, we have secured the necessary feature space. While currently unstructured and noisy, these specific columns were selected because they contain the latent behavioral

and environmental signals required for an algorithm to differentiate between minor and severe injuries.

Capturing the True Operational Baseline

A core data science goal is to build a model that performs reliably in a live, imbalanced production environment. Consequently, the data selection process intentionally ingested a broad, unstratified sample of all emergency room visits rather than artificially filtering for only severe cases. By selecting data that reflects the true ~10:1 class imbalance (where critical cases represent approximately 9% of the volume), we ensure the subsequent modeling phases will be trained to navigate the actual statistical reality of the CSIR pipeline.

1.2 Data Cleaning Process

1.2.1 Missing Data Treatment

Below table summarizes the list number and percentage of records with NULL and Unknown Data.

Column Name	Null Count	Null %	Unknown Count	Total_Missing(Null & Unknown)	Percentage Missing
Fire_Involvement	0	0.00	7283693	7283693	99.56
Other_Diagnosis_2	7214095	98.61	0	7214095	98.61
Diagnosis_2	6866693	93.86	0	6866693	93.86
Body_Part_2	6866693	93.86	0	6866693	93.86
Other_Race	6821041	93.24	0	6821041	93.24
Other_Diagnosis	6329255	86.52	0	6329255	86.52
Drug	5283928	72.23	0	5283928	72.23
Alcohol	5283928	72.23	0	5283928	72.23
Hispanic	5283928	72.23	0	5283928	72.23
Race	0	0.00	2145028	2145028	29.32
Location	0	0.00	2099509	2099509	28.70
Sex	0	0.00	978	978	0.01
Disposition	0	0.00	382	382	0.01

Treating NULL Values

Column Name	Null/Missing-Unknown?	Treatment Description
Other Diagnosis 2	NULL	93% of the records have a NULL value. Available only from 2019. This feature holds the description of the diagnosis. Fill the NULL values with the text "UNKNOWN"
Diagnosis 2	NULL	98% of the records have a NULL value. Only used with diagnosis code 71 - Other.
Body Part 2	NULL	94% of the records have a NULL value. Added to log the injury to the secondary body part. Defines the secondary body part code - fill it with '87' - Not stated/Un Known.
Other Race	NULL	Only used when Race = 3 (Other) and 93% of the values are null. This feature describes the Race as a text. For records with a NULL value, we can fill with the text "OTHER" and consider it as a separate class.

Other Diagnosis	NULL	Only used when the diagnosis is 71 - Other. 86% of the records have a NULL value. This feature is filled with description of the diagnosis. We can fill the NULL values with the text 'UNKNOWN' to consider it as a separate class.
Drug	NULL	Use of Drugs or Medication Contributed to the Incident or the Severity of the Injury - available from 2019. There are 2 categories of values in the field. 0 = No/No Information 1 = Yes Fill the NULL values with a new value "3" to consider it as a separate class.
Alcohol	NULL	Use of Drugs or Medication Contributed to the Incident or the Severity of the Injury - available from 2019. There are 2 categories of values in the field. 0 = No/No Information 1 = Yes Fill the NULL values with a new value "3" to consider it as a separate class.
Hispanic	NULL	There are three categories of observations 1. 0 - Unknown 2. 1 - Yes 3. 2 - No For records with NULL values, we can fill with a 0 (Unknown)

Treating UNKNOWN Values

Column Name	Null/Missing-Unknown?	Treatment Description
Fire Involvement	Unknown	99% of the records does not have a value - we can treat this as a category and will not substitute.
Race	Unknown	29% of the records does not have a value - we can treat this as a category and will not substitute.

Location	Unknown	28% of the records does not have a value - we can treat this as a category and will not substitute.
Sex	Unknown	Less than 1% of the values does not have a Sex. We can substitute these records with the common Sex value - Male (1)
Disposition	Unknown	Roughly 382 records do not have a disposition - we need to know the disposition to decide if a patient was hospitalized or not. we have two options. 1. Drop these records as the count is <1% 2. Classify these records as 'Not Hospitalized' when we perform the feature engineering. We will choose **#2** at this juncture.

Missing Values in the Age Feature

The Age feature contained missing values that required mathematical imputation to maintain algorithmic stability. To strictly prevent data leakage, a median imputation strategy was deployed post-partitioning. The median age was calculated exclusively derived from the Training subset. This static scalar value was subsequently applied to fill missing values within the Training, Validation, and Hold-Out test sets. This ensured that the evaluation partitions were cleansed without their underlying distributions influencing the imputation logic.

1.2.2 Outlier Detection and Handling

The outlier detection and handling need to be executed as a two-phase iterative approach to ensure data integrity across the CRISP-DM life cycle. We will initially perform the detection and cleaning using the raw data where we evaluate the data for physical impossibility and data entry errors. We will need to perform the outlier

detection and cleansing after the feature engineering phase to again make sure that there are no anomalies in the newly engineered features.

Categorical Outliers

Outlier detection was performed on categorical features and we identified the below exceptions.

- 147 products had fewer than 100 injuries and can be binned together as like “low-frequency-products”.
- Narratives for 10 records were less than 10 bytes. We should ignore the narratives of these 10 rows.

Numerical Outliers

- Age is the only numerical feature in the dataset, and this column needs transformation before checking for outliers. We will need to perform outlier detection on this feature after inconsistency resolution.

Date Outliers

- No outliers were found in the Treatment date feature.

1.2.3 Inconsistency Resolution

- Product Code Inconsistency
47 products show inconsistent reporting and show sudden death/birth and indicates reclassification to another product code or an expiry. We need to harmonize the product codes or bin the codes.

Impacted product codes:

[394, 556, 569, 574, 876, 897, 899, 1132, 1145, 1317, 1329, 1381, 1394, 1395, 1417, 1465, 1466, 1527, 1543, 1545, 1552, 1606, 1618, 1620, 1629, 1706, 1707, 1710, 1841, 1855, 1909, 3215,

4802, 5004, 5016, 5017, 5019, 5020, 5021, 5022, 5023, 5024, 5025, 5042, 5043, 5045, 5046]

The legacy product codes were reclassified into the modern codes to ensure longitudinal consistency. Refer to the official CPSC documentation (U.S. Consumer Product Safety Commission, 2018) that explains the complete crosswalk methodology for product remapping. Below is the summary of the action that we have taken for the 47 products that needs to be harmonized.

Product ID	Reclassified Product ID	Injury Year	Comments
394	394	2006- Present	New Product - Do not reclassify/bin
556	556	2005- Present	Current - No need to reclassify
569	Combine 514, 564 to 569	2005- Present	Combines 0514 and 0564
574	5016		New code added in 2008
876	Combine 865, 873, 876 to 897	2005 product	Expansion of 0806 along with 0865 & 0873. In 2006, merged with 0865 and 0873 to make 0897
897	5016		
899	5016		
1132	Combine 1332, 1383, 1306, 1317, 1318, 1356, 5004 to 5021 (Toy Vehicles)	2005 product	Expansion of 1108 along with 1130 & 1131. Deleted in 2005. In 2010, discontinued use of product codes 1332, 1383, 1306, 1317, 1318, 1356, and 5004,

			and merged into 5021.
1145	Replace 1855 with 1145	2006- Present	Name change. Replaces 1855
1317	5021	2005-2009	In 2010, merged with 5 other codes and replaced by 5021 and 5017. One->Many Taxonomy
1329	5023		
1381	1381	2010-2024	Current - No need to reclassify
1394	1394	2010-2024	Current - No need to reclassify
1395	1395	2010-2024	Current - No need to reclassify
1417	1417, 1465, 1466 to Decorative Yard Equipment	2005-2008	Incorporates part of 0124. Deleted in NEISS 12/94. Restored in NEISS 5/00. Split into 1465 and 1466 in 2009. One - Many Taxonomy - Convert to Decorative Yard Equipment
1465	Decorative Yard Equipment	2009-2024	Split from 1417
1466	Decorative Yard Equipment	2009-2024	Split from 1417
1527	1527		
1543	1552		
1545	1552		
1552	1552	2021-2024	Current - No need to reclassify
1606	1606	2005-2010	Effective 2011, for use in Trauma hospitals only

1618	1618	2005-Present	Current - No need to reclassify
1620	1620	2005- Present	Current - No need to reclassify
1629	1629	2005-Present	Current - No need to reclassify
1706	1706	2005-2010	Effective 2011, for use in Trauma hospitals only
1707	1707	2005-2010	Effective 2011, for use in Trauma hospitals only
1710	1710	2005-2010	Effective 2011, for use in Trauma hospitals only
1841	1841	2005-2017	Expansion of 1801 along with 1842 & 1843
1855	1145	2005 product	Deleted in 2005, and replaced with 1145
1909	1909	2005-2008	Effective 2011, for use in Trauma hospitals only
3215	3215		Expansion of 3204 along with 3258
4802	4802	2010-2024	Current - No need to reclassify
5004	1395		Effective January 1, 2010, there was a major revision of the “toy codes.” The product code 5004/Toys, not elsewhere classified was replaced by several new codes, including 1395/Toys, not specified and 1381/Toys, not elsewhere classified, along with the

			implementation of a number of new toy codes to capture and refine the coding of products formerly coded as 5004. Mapping to the general Toys product.
5016	5016	2009-2024	Current - No need to reclassify
5017	Off-Road Vehicles (ATV/UTV)		
5019	Off-Road Vehicles (ATV/UTV)		
5020	Off-Road Vehicles (ATV/UTV)		
5021	Off-Road Vehicles (ATV/UTV)		
5022	Micromobility (Scooters/Hoverboards)		
5023	Micromobility (Scooters/Hoverboards)		
5024	Micromobility (Scooters/Hoverboards)		
5025	Micromobility (Scooters/Hoverboards)		
5042	5022		
5043	Off-Road Vehicles (ATV/UTV)		
5045	Power-Assisted Cycles (E-bikes)		
5046	Power-Assisted Cycles (E-bikes)		

- Age Vs Product Inconsistency

There are around 776 suspicious cases of infants using adult products. As the number of such records are very minimal – we can delete these records.

1.2.4 Error Correction

Unit of Measure Standardization (Infant Age):

The raw NEISS Age variable exhibited a structural inconsistency by combining two distinct units of measurement within a single numeric vector. While patients aged two years and older are recorded in integer years, infants under two years are recorded in months, prefixed with a "2" (e.g., a 6-month-old is coded as 206). To resolve this inconsistency and prevent machine learning algorithms from interpreting infants as >200-year-old outliers, a programmatic transformation was applied. All values of age greater than 200 were isolated, the prefix was stripped, (by subtracting 200 from the age) and the remaining months were converted into fractional years (divided by 12 months). This harmonized the entire column into a single, continuous unit of measurement (Years). Refer the Age section in Neiss Coding Manual January 2025(Rev.1). (n.d.). for more details.

Duplicate Check:

A duplicate check was performed on the dataset to confirm that the injury data is not duplicated in the dataset. No duplicate data was observed in the dataset – No correction applied.

1.2.5 Impact Assessment

The Data cleaning process reduced dataset from 7315732 to 7314956 records (0.01 % loss). Most of the data in the NEISS dataset is already clean and the only scenario that needed the amputation was due to a data inconsistency that we observed when we compared the patient age with the product involved in the injury.

1.3 Feature Engineering

To ensure structural parity across all data partitions while strictly preventing data leakage, feature engineering was bifurcated. Row-independent derivations (e.g., Hospitalization, Data Year) were applied globally prior to partitioning. Conversely, dataset-dependent transformations, such as numeric scaling and categorical encoding, were strictly fitted exclusively on the Training partition. The resulting transformation parameters were subsequently applied blindly to the Validation and Hold-Out partitions, preserving the pristine, unseen nature of the evaluation data.

1.3.1 Derived Features

The below features were derived from the original dataset

True Alcohol

The Alcohol feature in the dataset is only available from 2019 onwards and hence more than 72.23% unknown value. This field is added to derive any insights from the Narrative attribute indicating the usage of Alcohol before or during the occurrence of the injury.

Methodology:

- Text-Mining for Substance Usage: Parsed the text within the Narrative feature for specific alcohol-related keywords to construct a preliminary Alcohol_Narrative Boolean flag.
- Composite Feature Generation: Applied a logical inclusive-OR operation between the newly derived Alcohol_Narrative flag and the baseline Alcohol variable to engineer a master True_Alcohol feature, ensuring all documented instances of alcohol involvement were captured.

Alcohol Keywords:

'ALCOHOL', 'ETOH', 'INTOXICATED', 'INEBRIATED', 'DRUNK',
'SOBER', 'BAC ', 'BEER', 'WINE', 'LIQUOR', 'VODKA', 'WHISKEY',
'WHISKY', 'TEQUILA', 'RUM', 'GIN', 'BRANDY', 'CHAMPAGNE',
'MOONSHINE', 'DRINKING', 'PARTY', 'BAR', 'PUB', 'CLUB', 'TAVERN',
'HAPPY HOUR', 'KEG'

True Drug

The Drug feature in the dataset is only available from 2019 onwards and hence more than 72.23% unknown value. This field is added to derive any insights from the Narrative attribute indicating the substance usage of before or during the occurrence of the injury.

Methodology:

- Text-Mining for Substance Usage: Parsed the text within the Narrative feature for specific drug-related keywords to construct a preliminary Drug_Narrative Boolean flag.
- Composite Feature Generation: Applied a logical inclusive-OR operation between the newly derived Drug_Narrative flag and the baseline Drug variable to engineer a master True_Drug feature, ensuring all documented instances of drug involvement were captured.

Drug Keywords:

'DRUG', 'OVERDOSE', 'MEDICATION', 'PILL', 'TABLET', 'CAPSULE',
'INGEST', 'SUBSTANCE', 'NARCOTIC', 'OPIOID', 'OPIGTE',
'COCAINE', 'HEROIN', 'METHAMPHETAMINE', 'MARIJUANA',
'CANNABIS', 'FENTANYL', 'ECSTASY', 'MDMA', 'LSD ', 'PCP ',
'HASHISH', 'XANAX', 'VALIUM', 'PERCOCET', 'VICODIN',
'OXYCODONE', 'OXYCONTIN', 'ADDERALL', 'RITALIN', 'TYLENOL',

'ASPIRIN', 'IBUPROFEN', 'ADVIL', 'MOTRIN', 'BENADRYL', 'INSULIN',
'VITAMIN', 'MORPHINE', 'CODEINE'

Treatment Month

Derive this feature from the Treatment Date Feature

1.3.2 Transformations

Transformation applied on Categorical Features

Product_1:

The primary independent variable, Product_1, exhibits a high cardinality (837 unique discrete values). To prevent the Curse of Dimensionality and computational degradation associated with standard One-Hot Encoding, a Target Encoding (Mean Encoding) strategy is employed. The categorical product codes were transformed into continuous numeric variables representing their historical probability of the target class (Hospitalized). To strictly prohibit target leakage, the encoding probabilities were fitted exclusively on the Training partition, utilizing statistical smoothing to penalize low-frequency categories, before being mapped to the Validation and Hold-Out datasets.

Diagnosis, Body Part, Location, Body Part 2, EMS/Severity Escalation, Safety Equipment Utilization, Third-Party Intervention, Height Risk, and Water Risk

One-Hot Encoding was applied to the nominal categorical features to prevent the algorithms from inferring false ordinal relationships. Because these variables have no inherent mathematical ranking, transforming them into distinct binary vectors (0s and 1s) is a mathematical prerequisite. This ensures the predictive model evaluates the distinct presence or absence of each category without falsely associating magnitude or hierarchy. We chose one hot encoding due to the low cardinality of these features.

Transformation applied on Numeric Features

The continuous numerical feature 'Age' was transformed using a Standard Scaler (Z-score normalization) to enforce a mean of 0 and a standard deviation of 1. This standardization ensures that the feature's original magnitude does not disproportionately influence spatial calculations. It is a critical mathematical prerequisite for distance-based machine learning algorithms—such as K-Nearest Neighbors or Support Vector Machines—which rely on Euclidean or Manhattan metrics and are highly sensitive to unscaled variances.

1.3.3 Temporal Features

Treatment Year

The treatment year has been derived from the treatment date.

Treatment Month

The treatment month has been derived from the treatment date.

1.3.4 Domain Specific Features

EMS/Severity Escalation, Safety Equipment Utilization, Third-Party Intervention, Height Risk, and Water Risk

To augment the predictive capacity of the baseline categorical variables, rule-based Natural Language Processing (NLP) was applied to the unstructured clinical narratives. A series of deterministic regular expression (Regex) algorithms were developed to extract high-value contextual vectors that are historically under-coded in the CPSC taxonomy. This yielded five distinct binary features: EMS/Severity Escalation, Safety Equipment Utilization, Third-Party Intervention, Height Risk, and Water Risk. These boolean indicators provide the classification algorithms with critical micro-environmental context necessary for accurately predicting hospitalization probability.

Injury Mechanism:

To capture the root behavioral cause of the incidents, a new categorical feature, Injury_Mechanism, was engineered from the unstructured Narrative column. A deterministic Regular Expression (Regex) algorithm was deployed to scan the 142-character text vectors for standardized keyword groups, effectively categorizing raw clinical text into five distinct quantitative mechanisms (e.g., Fall, Burn, Ingestion). Narratives lacking these specific keyword patterns were binned as 'Other/Unspecified' to maintain dataset dimensionality without introducing speculative bias.

True Alcohol

The 'Alcohol' flag indicates whether a patient consumed alcohol in relation to their incident. Because this dedicated variable was only introduced to the dataset in 2019, approximately 73% of historical records are structurally missing this information. However, prior to 2019, hospital staff frequently documented alcohol or substance involvement directly within the unstructured 'Narrative' feature using common clinical keywords. By applying text-mining techniques to extract these keywords, we can retroactively derive and impute the missing alcohol flags. Recovering this critical behavioral data will empower the machine learning algorithms to mathematically isolate correlations between alcohol consumption and overall injury severity.

True Drug

The 'Drug' flag indicates whether a patient consumed substance/drug in relation to their incident. Because this dedicated variable was only introduced to the dataset in 2019, approximately 73% of historical records are structurally missing this information. However, prior to 2019, hospital staff frequently documented drug/substance involvement directly within the unstructured 'Narrative' feature using common clinical keywords. By applying text-mining techniques to extract these keywords, we can retroactively derive and impute the missing drug flags. Recovering this critical behavioral data will empower the machine learning algorithms to mathematically isolate correlations between substance consumption/abuse and overall injury severity.

Treatment Year and Treatment Year

Temporal variables, specifically 'Treatment_Year' and 'Treatment_Month', were extracted from the raw 'Treatment_Date'

feature to capture two distinct chronological patterns. The 'Month' variable enables the algorithm to identify cyclical, seasonal behaviors in product-related injuries, while the 'Year' variable contextualizes macro-level historical anomalies, such as the dramatic shifts in injury rates documented during the COVID-19 pandemic.

EMS Severity Flag

Extracting the 'EMS/Severity' flag from the unstructured narratives provides a critical proxy for initial incident perception. By capturing the specific mode of medical transport—such as distinguishing a critical helicopter airlift from a standard ambulance response—we engineer a feature that inherently signals trauma level. This allows the predictive algorithms to mathematically weight these emergency escalation events when correlating incident details with the final injury severity.

Safety Gear Flag

Extracting a 'Safety Gear' flag from the unstructured narratives introduces a critical mitigating variable to contextualize the baseline trauma. For instance, sustaining a head impact while wearing a helmet typically results in a distinctly different clinical trajectory compared to an unprotected incident. Integrating this feature allows the classification algorithms to mathematically weigh the protective effect of safety equipment when predicting the patient's final clinical disposition.

Third Party Flag

Extracting the third-party flag from the narrative also plays a critical role in understanding and predicting the nature and severity of injury – it indicates if a third person was involved in the injury.

Height Risk Flag

Extracting the height risk flag from the narrative also plays a critical role in understanding and predicting the nature and severity of injury. This flag helps us in understanding if the injury was caused due to a fall from a height.

Water Risk Flag

Extracting the Water risk flag from the narrative also plays a critical role in understanding and predicting the nature and severity of injury. This flag helps us in understanding if the injury was caused due to an involvement with water/bodies.

Hospitalization

We will derive the Target Variable Hospitalized from the Disposition code Hospitalized

Disposition Code	Hospitalized (Yes/No)
4,2,5,8	Yes
1,6,9	No

1.4 Data Integration and Final Preparation

1.4.1 Integration Approach

This project employed a single-source methodology utilizing only the NEISS database. External data integration was excluded from the project scope, as the native incident records provided sufficient dimensionality for predictive modeling. The integration effort was therefore strictly internal, focusing on appending newly engineered features—derived from text-mining the clinical narratives—into the core dataset to enrich the model's predictive context.

1.4.2 Data Formatting

Type Casting: Converted raw Treatment Date into standard Python datetime objects to allow for precise temporal slicing.

String Standardization:

Forced all 142-character narratives into uppercase formatting to ensure the Regex extraction algorithms were case-insensitive and mathematically stable.

Categorical Binarization:

Applied One-Hot Encoding to reformat nominal text variables into algorithm-compatible binary matrices (0/1), satisfying the strict numeric input requirements of the predictive models.

Target Encoding was used on the Product_1 field due to high cardinality. Structured data in wide format with one row per injury case and columns for each feature, ensuring all numeric features are float type, categorical variables are properly encoded, and target variable is properly encoded as binary 0/1

1.4.3 Train/Validation/Test Splitting

Used 70/15/15 (for the Train/Evaluation/Holdout) split with stratification on the composite target variable (Year and Hospitalized) to ensure equal class representation across sets, with temporal ordering preserved to simulate real-world prediction scenarios.

1.4.4 Statistical Variable Selection (Phase 2)

Following the completion of feature engineering, we evaluated the statistical relationship between the available predictors and the target variable ('Hospitalized'). Initially, a Chi-Square test of independence was applied to assess statistical significance. However, due to the extreme sensitivity of the Chi-Square statistic to large sample sizes, trivial differences yielded highly significant p-values across all categorical variables, rendering the test ineffective for feature selection. To overcome this limitation, we utilized Cramer's V to quantify the actual effect size—or strength of association—between each predictor and the target. This metric provided a reliable, scale-independent threshold for isolating the most impactful features for the subsequent modeling phase.

Target Variable: Hospitalized

The below features were selected for modelling as predictors:

1. Age
2. Diagnosis
3. Body Part
4. Product 1
5. Injury Mechanism
6. Location
7. Body Part 2

1.4.5 Final Dataset Characteristics

The final dataset provided to the machine learning algorithms represents a highly optimized, signal-dense feature space. To arrive at these final characteristics, a secondary, post-engineering selection phase was conducted on the fully enriched dataset. We applied Chi-Square Tests of Independence and Cramér's V calculations as a final statistical gatekeeper across both native and engineered features. Consequently, the final dataset excludes all low-impact noise and retains only the variables—such as the engineered EMS/Severity flag and the native Age feature—that mathematically demonstrated a significant and robust correlation with the target patient disposition. The final dataset contains 7,314,956 rows and 99 features.

Hospitalized?	Count	Ratio
0	6,650,119	91
1	664,837	9

The dataset is severely imbalanced with 91:9 for the target variable.

Record Count (Train/Evaluation/Holdout)

Below is the split of the number of records between train, evaluation and holdout datasets.

Dataset	Record Count	Percentage	Feature Count
Train	5,120,468	70%	99
Evaluation	1,097,244	15%	99
Holdout	1,097,244	15%	99
Total	7,314,956	100%	99

Count of Data by Target Variable

Dataset	Hospitalized?	Count	Ratio
Train	0	4655082	90.91
Train	1	465386	9.09
Evaluation	0	997519	90.91
Evaluation	1	99725	9.09

Holdout	0	997518	90.91
Holdout	1	99726	9.09

We can see that the data is split in a stratified manner by target variable.

Temporal Stratified Record Volume

Treatment		
Year	Hospitalized	Count
2005	0	236967
2005	1	15254
2006	0	238747
2006	1	15748
2007	0	242403
2007	1	16439
2008	0	244657
2008	1	17293
2009	0	254571
2009	1	19747
2010	0	263406
2010	1	20564
2011	0	257017
2011	1	20500
2012	0	254802
2012	1	21240
2013	0	241922
2013	1	21896
2014	0	236169
2014	1	21056
2015	0	229419
2015	1	21950
2016	0	238688
2016	1	23927
2017	0	244791
2017	1	26011
2018	0	225755
2018	1	27390
2019	0	223061

2019	1	28016
2020	0	187255
2020	1	29286
2021	0	208390
2021	1	29897
2022	0	198672
2022	1	27652
2023	0	207123
2023	1	29635
2024	0	221267
2024	1	31885

We can see that the data is split in a stratified manner by treatment year and target variable.

Feature List in the Final Dataset

Feature List
cat_Diagnosis_41
cat_Diagnosis_42
cat_Diagnosis_46
cat_Diagnosis_47
cat_Diagnosis_48
cat_Diagnosis_49
cat_Diagnosis_50
cat_Diagnosis_51
cat_Diagnosis_52
cat_Diagnosis_53
cat_Diagnosis_54
cat_Diagnosis_55
cat_Diagnosis_56
cat_Diagnosis_57
cat_Diagnosis_58
cat_Diagnosis_59
cat_Diagnosis_60
cat_Diagnosis_61
cat_Diagnosis_62
cat_Diagnosis_63
cat_Diagnosis_64
cat_Diagnosis_65

cat_Diagnosis_66
cat_Diagnosis_67
cat_Diagnosis_68
cat_Diagnosis_69
cat_Diagnosis_71
cat_Diagnosis_72
cat_Diagnosis_73
cat_Diagnosis_74
cat_Body_Part_0
cat_Body_Part_30
cat_Body_Part_31
cat_Body_Part_32
cat_Body_Part_33
cat_Body_Part_34
cat_Body_Part_35
cat_Body_Part_36
cat_Body_Part_37
cat_Body_Part_38
cat_Body_Part_75
cat_Body_Part_76
cat_Body_Part_77
cat_Body_Part_79
cat_Body_Part_80
cat_Body_Part_81
cat_Body_Part_82
cat_Body_Part_83
cat_Body_Part_84
cat_Body_Part_85
cat_Body_Part_87
cat_Body_Part_88
cat_Body_Part_89
cat_Body_Part_92
cat_Body_Part_93
cat_Body_Part_94
cat_Location_0
cat_Location_1
cat_Location_2
cat_Location_4
cat_Location_5
cat_Location_6
cat_Location_7

cat_Location_8
cat_Location_9
cat_Body_Part_2_0.0
cat_Body_Part_2_30.0
cat_Body_Part_2_31.0
cat_Body_Part_2_32.0
cat_Body_Part_2_33.0
cat_Body_Part_2_34.0
cat_Body_Part_2_35.0
cat_Body_Part_2_36.0
cat_Body_Part_2_37.0
cat_Body_Part_2_38.0
cat_Body_Part_2_75.0
cat_Body_Part_2_76.0
cat_Body_Part_2_77.0
cat_Body_Part_2_79.0
cat_Body_Part_2_80.0
cat_Body_Part_2_81.0
cat_Body_Part_2_82.0
cat_Body_Part_2_83.0
cat_Body_Part_2_84.0
cat_Body_Part_2_85.0
cat_Body_Part_2_87.0
cat_Body_Part_2_88.0
cat_Body_Part_2_89.0
cat_Body_Part_2_92.0
cat_Body_Part_2_93.0
cat_Body_Part_2_94.0
cat_Injury_Mechanism_Burn
cat_Injury_Mechanism_Fall
cat_Injury_Mechanism_Ingestion/Choking
cat_Injury_Mechanism_Laceration/Cut
cat_Injury_Mechanism_Other/Unspecified
cat_Injury_Mechanism_Struck By
remainder_Product_1
remainder_Age_Scaled

1.4.6 Preparation Pipeline

To guarantee reproducibility and prevent data leakage, the data preparation process was structured as a strict, sequential pipeline. First, programmatic text-mining (via Regex) and temporal extraction were applied across the entire dataset to engineer key proxy variables, such as EMS/Severity, Safety Gear, and Treatment Year. Following feature construction, Chi-Square and Cramer's V tests were conducted to evaluate statistical significance. Given the massive scale of the dataset, the Cramer's V statistic was specifically utilized to measure the strength of association between all predictors and the target variable, allowing us to isolate only the most highly correlated features for modeling. This refined feature set was then partitioned into Training, Validation, and Hold-Out Test sets using a stratified split based on both Treatment Year and the Target variable (Hospitalized). Finally, a formal transformation pipeline was built using the scikit-learn and imblearn libraries. This pipeline sequentially executed One-Hot Encoding for nominal categories, Standard Scaling (Z-score normalization) for the continuous 'Age' variable, and Random Under sampling to balance the target classes—crucially applying this under sampling exclusively to the training partition to protect the integrity of the test sets.

2. Modelling

2.1 Modelling Strategy

2.1.1 Problem Framing

This is a binary classification problem to predict if a patient will be Hospitalized (based on the disposition code) as this directly addresses the business goal of identifying factors associated with hospitalization for product-related injuries.

2.1.2 Algorithm Selection Rationale

Baseline Model

- Selected logistic regression for interpretability.

Primary Model(s)

- Decision Tree for interpretability and to capture non-linear relationships.
- Random forest to capture complex non-linear relationships and to prevent overfitting that happens with Decision Trees.
- Gradient boosting (XGBoost) for maximum predictive performance, allowing comparison between simple/complex approaches

2.1.3 Evaluation Strategy

We will use 70/15/15 split on the Train/Evaluation/Holdout datasets, and we will use a combination of PR-AUC (Precision-Recall Area Under the Curve) and the Minority F1-Score. Because the Composite Temporal Stratification methodology perfectly preserved the inherent class skew of the target disposition, traditional evaluation metrics like ROC-AUC risk presenting an overly optimistic assessment due to the dominance of the negative majority class (not Hospitalized). Consequently, the primary evaluation metrics selected were the

Precision-Recall Area Under the Curve (PR-AUC) and the Minority Class F1-Score. By isolating the algorithm's performance on the True Positives and False Positives, these metrics strictly penalize the model for failing to identify the rare, severe trauma cases, providing a mathematically robust evaluation of its real-world clinical utility.

2.1.4 Computational Considerations

Working with a large dataset on a standard laptop, so will use a 10% random sample for initial model development and testing before running final models on the complete dataset

2.1.5 Connection to Data Science Goals and Success Criteria

Enabling Complex Signal Extraction

Raw emergency room triage data is inherently noisy, consisting of disjointed medical codes and unstructured text. The data science goal of discovering predictive patterns required transforming this raw input into a mathematically viable feature space. Through rigorous data cleaning, categorical encoding, and feature engineering, the dataset has been structured to expose potential non-linear relationships—such as the interaction between specific patient demographics and product categories—allowing advanced algorithms to effectively search for predictive signals.

Grounding the Imbalance Strategy

A core technical objective is building a model that survives the reality of a severe class imbalance, where the target priority cases represent only ~9.1% of the dataset. Rather than prematurely distorting this reality through aggressive synthetic oversampling during the preparation phase, the true 10:1 distribution was preserved and carefully stratified across the training and testing splits. This ensures

that the upcoming models will learn and be evaluated against the true operational prevalence they will encounter in production.

Establishing the Evaluation Framework

By quantifying the exact nature of the class imbalance during the preparation phase, the data science goal of selecting the correct evaluation metric has been firmly established. It mathematically justifies the strategic decision to discard overall "Accuracy" for the upcoming Modeling phase. The prepared dataset dictates that model success must instead be measured by Precision-Recall curves and the F1-score, ensuring the algorithms are penalized for false positives (alert fatigue) and false negatives (missed safety events).

2.2 Model Development

2.2.1 Baseline Model

Rationale and Model Selection

To establish a performance baseline, a standard Logistic Regression model was developed. Logistic Regression was selected as the baseline algorithm due to its high interpretability, computational efficiency, and ability to output probabilistic predictions rather than strict class labels. Establishing this baseline provides a benchmark to measure the added value of more complex, non-linear models later in the development phase.

Evaluation Metrics given the nature of the target variable and the business objectives the overall accuracy is an insufficient metric. Instead, the model was evaluated using the following metrics to better capture the trade-off between False Positives and False Negatives: F1-Score (Primary Metric), Precision, Recall (Sensitivity), ROC-AUC & AP Score.

Initial Performance (Default Threshold)

The Logistic Regression model was initially trained and evaluated using Scikit-Learn's default decision threshold of 0.50. Under this default configuration, the model achieved the following performance on the test set on the minority class.

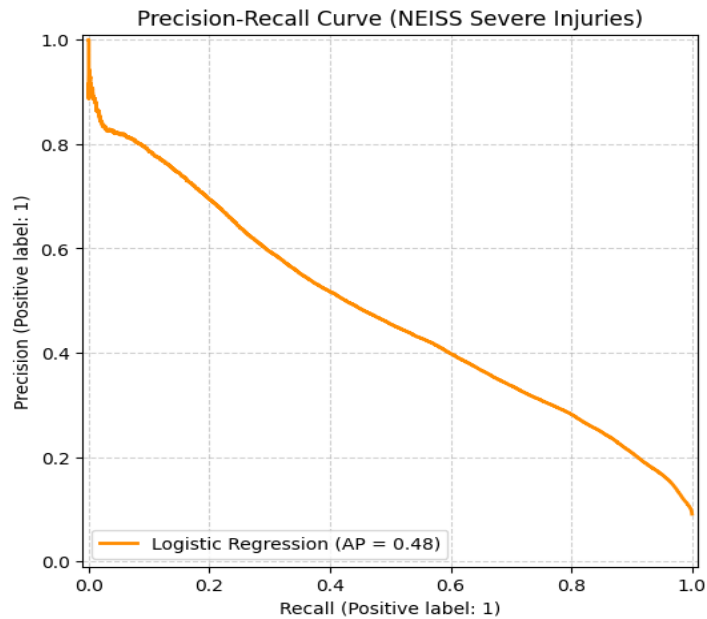
F1-Score: 0.41

Precision: 0.27

Recall: 0.81

ROC-AUC: 0.876

Average Precision AP: 0.48



While the ROC-AUC score indicated that the underlying model had predictive power, the default 0.50 threshold yielded a suboptimal F1-score. The model was leading to poor precision / under-predicting the positive /minority class.

Threshold Optimization and Parameter Tuning

Because the standard 0.50 cutoff is arbitrary and rarely optimal for imbalanced datasets, the decision threshold was treated as a tunable parameter. By analyzing the Precision-Recall/AUC Curve, the probability threshold was optimized to maximize the F1-score.

The optimal probability threshold was identified as 0.73.

A custom Scikit-Learn wrapper was implemented to embed this custom decision boundary directly into the prediction pipeline.

Refined Model Performance (Adjusted Threshold)

Applying the customized 0.73 decision threshold significantly improved the model's alignment with the project's evaluation criteria.

By requiring a higher predicted probability (73% certainty) before

classifying an instance as positive, the model successfully balanced the precision-recall trade-off.

The updated evaluation metrics using the 0.73 threshold are:

```
--- Optimization Results ---  
Optimal Probability Threshold: 0.7390  
Maximum F1-Score Achieved:      0.4826  
Corresponding Precision:        0.4188  
Corresponding Recall:           0.5694  
  
--- New Optimized Confusion Matrix ---  
[[918696  78823]  
 [ 42937  56788]]
```

Refined F1-Score: 0.48 (An improvement of +7.00)

Refined Precision: 0.42

Refined Recall: 0.57

This optimized Logistic Regression model now serves as the official baseline against which subsequent, more complex algorithms will be compared.

2.2.2 Primary Models

Model-1: Non-Linear Classification via Decision Tree

Algorithm Description and Parameters

To capture potential non-linear interactions between predictor variables and the target variables, a CART (Classification and Regression Trees) Decision Tree algorithm was deployed alongside the baseline Logistic Regression model.

A Decision Tree classifier was applied to the NEISS dataset to predict the target variable (Class 1). The initial evaluation yielded an overall accuracy of 0.79 (79%). However, due to a severe class imbalance in the test set—where the majority class (Class 0) outnumbers the minority class (Class 1) by a ratio of approximately 10:1—accuracy is a highly misleading metric.

The Decision Tree's lower accuracy of 0.79 is a direct result of its attempt to identify the minority class rather than ignoring it, necessitating a deeper look at the confusion matrix and class-specific metrics.

Evaluation Metrics (Minority Class Focus)

To properly assess the model's utility, the evaluation focused heavily on its performance regarding the positive class (Class 1), utilizing Precision, Recall, and the F1-score.

Recall (0.82): The model demonstrates a strong ability to detect the positive class, successfully identifying 82% of all actual Class 1 instances (81,411 True Positives vs. 18,314 False Negatives). This indicates that the model is highly sensitive to the minority class.

Precision (0.28): The high recall comes at a significant cost to precision. The model generated 212,581 False Positives. Consequently, when the model predicts an instance belongs to Class 1, it is only correct 28% of the time.

F1-Score (0.41): As the harmonic mean of Precision and Recall, the F1-score of 0.41 reflects the severe penalty incurred by the high False Positive rate.

While the Decision Tree successfully detects the minority class, the resulting F1-score indicates that the current precision-recall trade-off is suboptimal. To make the model viable for deployment, future iterations need to focus on reducing the False Positive rate.

Training Process and Challenges

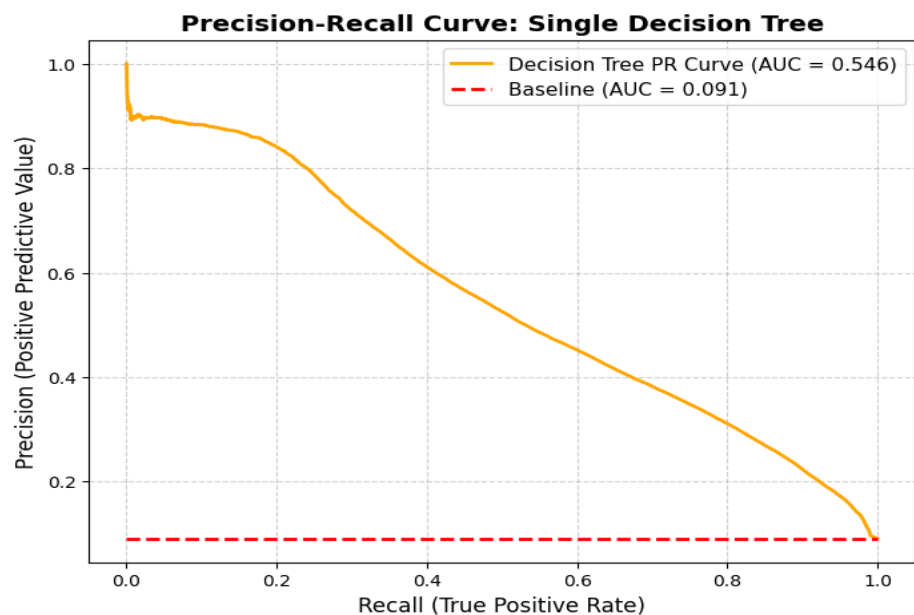
The training partition was balanced via Random Under sampling within an imblearn pipeline to isolate the algorithmic learning process from the Evaluation set. To mitigate the algorithm's inherent susceptibility to high variance and overfitting, structural regularization hyperparameters were enforced, specifically capping the maximum tree depth and establishing a minimum leaf node threshold.

Hyperparameter Tuning Approach

During the hyperparameter tuning phase via RandomizedSearchCV, the Decision Tree exhibited a highly polarized performance profile (Precision: 0.82, Recall: 0.30, F1-Score: 0.43). This behavior is directly attributable to the default classification mechanics of the evaluation library. When optimizing for the F1-score across cross-validation folds, the search algorithm evaluates model performance using the default prediction method, which enforces a strict, static 0.50 probability threshold.

As the NEISS dataset contains a severe class imbalance, achieving a high F1-score at this uncalibrated cutoff mathematically forced the search algorithm to select overly restrictive hyperparameters (e.g., heavily constrained tree depth or high minimum sample requirements for leaf nodes). This diagnostic finding reinforces the methodological conclusion that for highly imbalanced datasets, calibrating the decision threshold post-training is vastly superior to forcing the algorithm to adapt its physical architecture to a default 0.50 cutoff.

The PR-AUC of 0.54 tells a different story. It reveals that the Decision Tree *can separate* the classes well—it is performing vastly better than the ~0.09 random guessing baseline.



The high PR-AUC means that if you had moved away from the default 0.50 threshold and looked at the rest of the tree's

probability curve (for instance, dropping the threshold to 0.15 or 0.20), we would have found a completely different point on that stepped graph where the Recall jumps up significantly without entirely destroying the Precision.

Below are the hyperparameters that were selected for optimal f1 score:

Max Tree Depth: 13

Minimum Sample Split: 5

Minimum Sample Leaf: 11

Measurement Criterion: gini

Training Performance Metrics

Initial Metric:

Precision: 0.28

Recall: 0.82

F1 Score: 0.41

After Hyperparameter tuning:

F1 Score: 0.50

Model-2: Ensemble Classification via Random Forest

Algorithm Description and Parameters

To address the variance and overfitting vulnerabilities inherent in standalone CART algorithms, a Random Forest ensemble classifier was implemented. By aggregating the predictions of 100 independent, shallow decision trees trained on bootstrapped data subsets, the model achieved higher predictive stability. Class imbalance was neutralized during the .fit() execution using an imblearn Random Under sampling

pipeline. Structural hyperparameters, including max_depth and min_samples_leaf, were constrained to prevent the component estimators from capturing stochastic noise, prioritizing real-world generalizability on the Validation set.

Rationale and Ensemble Approach

Following the evaluation of the single Decision Tree, a Random Forest Classifier was implemented. As an ensemble learning method, the Random Forest aggregates the predictions of multiple decision trees trained on random subsets of the data. The objective of introducing this model was to reduce the high variance and potential overfitting observed in the single Decision Tree, theoretically improving precision on the unseen validation set.

Performance Metrics (Validation Set)

The Random Forest was evaluated on the validation set, focusing heavily on the minority class (Class 1) to account for the ~10:1 class imbalance.

Recall: 0.76 (Successfully identified 76,213 out of 99,725 true positive instances)

Precision: 0.26 (Generated 219,311 False Positives)

F1-Score: 0.39

Comparative Analysis (Random Forest vs. Decision Tree)

Contrary to typical expectations, the ensemble model did not yield an improvement over the single Decision Tree baseline.

The Random Forest experienced a drop in both Recall (from 0.82 to 0.76) and Precision (from 0.28 to 0.26), resulting in a lower F1-score (0.39 vs. 0.41). This performance degradation highlights a key dynamic of the dataset's class imbalance. The variance-reduction mechanism inherent to Random Forests (bagging) smooths the decision boundaries. While this prevents overfitting, in a heavily imbalanced environment, this smoothing naturally biases the aggregated "vote" toward the overwhelmingly dominant majority class (Class 0). The ensemble essentially averaged out the hyper-specific rules the single Decision Tree had utilized to identify the minority class.

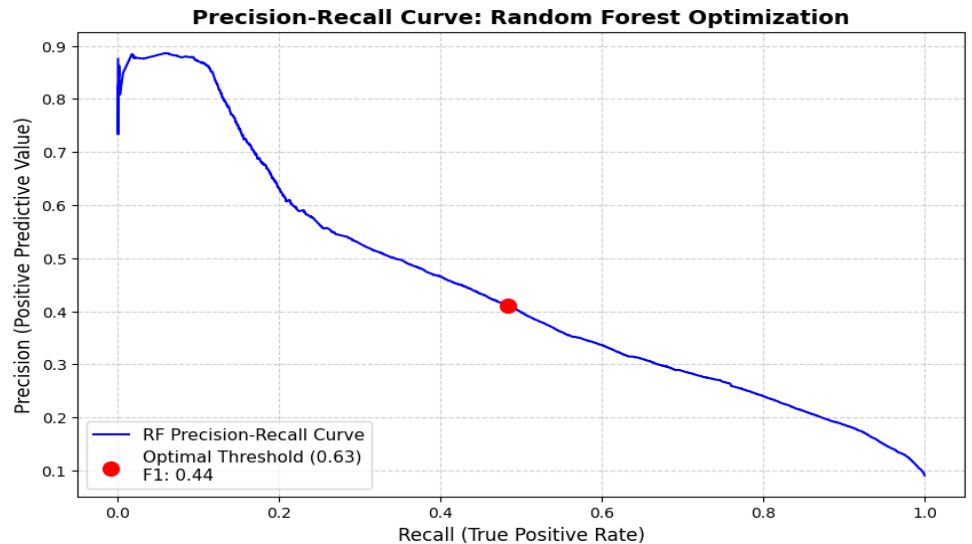
Training Process and Challenges

The training partition was balanced via Random Under sampling within an imblearn pipeline to isolate the algorithmic learning process from the Evaluation set. To mitigate the algorithm's inherent susceptibility to high variance and overfitting, structural regularization hyperparameters were enforced, specifically capping the maximum tree depth and establishing a minimum leaf node threshold.

Hyperparameter Tuning Approach

To address the severe class imbalance within the NEISS dataset, the Random Forest model's optimization strategy focused entirely on calibrating the decision threshold rather than altering structural hyperparameters. The default ensemble model, while effective at reducing variance, yielded a suboptimal F1-score due to an uncalibrated 0.50 probability cutoff that generated an excessive number of False Positives. To correct this, a post-training threshold optimization was

conducted. The models. predict_proba() outputs were extracted, and a Precision-Recall Curve was utilized to evaluate the trade-off between false alarms and missed classifications across all potential thresholds. By systematically



calculating the F1-score at each cutoff point, the exact mathematical probability threshold that maximized the F1-score was identified. This targeted approach successfully shifted the model's decision boundary, significantly reducing the volume of False Positives and elevating overall precision while preserving the necessary recall for the minority target variable.

Training Performance Metrics

Performance Metrics before Hyperparameter Tuning

Recall: 0.76

Precision: 0.26

F1 Score: 0.39

Performance After Hyperparameter Tuning

Optimal Threshold: 0.76

Recall: 0.48

Precision: 0.41

F1 Score: 0.44

Model-3 Gradient Boosting Via XG Boost

Algorithm Description and Parameters

To maximize predictive accuracy and capture complex multivariate interactions, an extreme Gradient Boosting (XGBoost) classifier was deployed. Unlike parallel ensemble methods, XGBoost utilizes sequential additive training, optimizing a logarithmic loss function via gradient descent to iteratively correct residual errors. Class imbalance was resolved pre-training using an imblearn Random Under sampling pipeline. To prevent overfitting inherent to gradient boosting, strict regularization constraints were applied, specifically capping individual tree complexity (max_depth=5) and penalizing aggressive weight updates (learning rate=0.1).

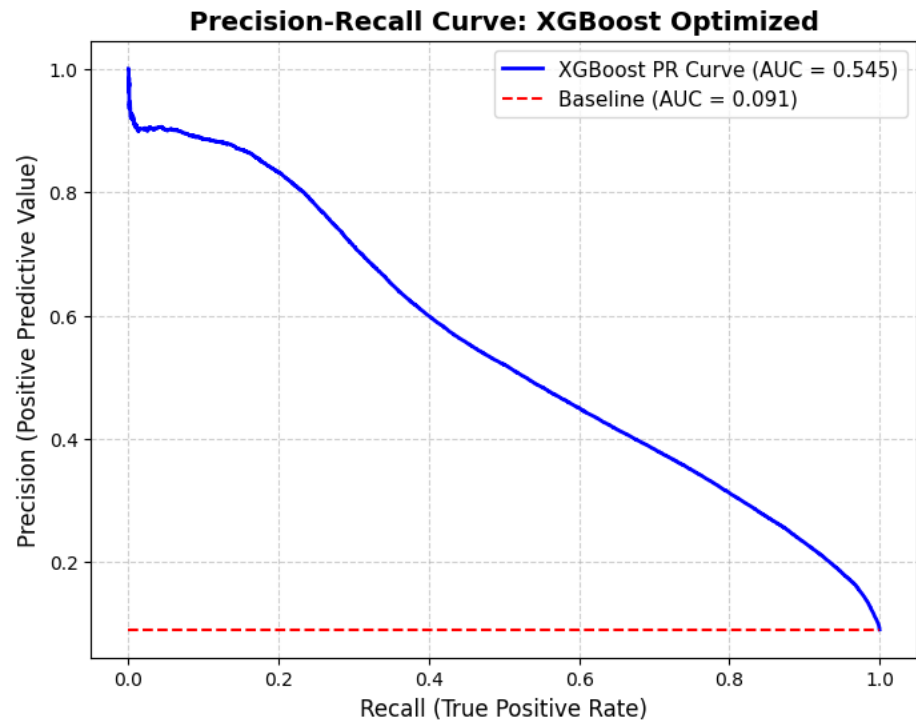
Training Process and Challenges

The training partition was balanced via Random Under sampling within an imblearn pipeline to isolate the algorithmic learning process from the Evaluation set.

Hyperparameter Tuning Approach

To address the severe class imbalance within the NEISS dataset, the XGBoost model's optimization strategy focused entirely on calibrating the decision threshold rather than altering structural hyperparameters. The default ensemble

model, while effective at reducing variance, yielded a suboptimal F1-score due to an uncalibrated 0.50 probability cutoff that generated an excessive number of False Positives. To correct this, a post-training threshold optimization was conducted. The models. predict_proba() outputs were extracted, and a Precision-Recall Curve (XGBoosting. (n.d.). *Evaluate XGBoost performance with precision-recall curve.*) was utilized to evaluate the trade-off between false alarms and missed classifications across all potential thresholds.



By systematically calculating the F1-score at each cutoff point, the exact mathematical probability threshold that maximized the F1-score was identified. This targeted approach successfully shifted the model's decision boundary, significantly reducing the volume of False Positives and

elevating overall precision while preserving the necessary recall for the minority target variable.

To comprehensively evaluate the optimized XGBoost model's performance across all classification thresholds, a Precision-Recall (PR) Curve was generated. Because the NEISS dataset exhibits a severe ~10:1 class imbalance, standard Receiver Operating Characteristic (ROC) curves can present overly optimistic visualizations. The PR curve directly plots the trade-off between Positive Predictive Value (Precision) and True Positive Rate (Recall), isolating the evaluation strictly to the minority target class.

Training Performance Metrics

Performance Metrics before Hyperparameter Tuning

Recall: 0.82

Precision: 0.29

F1 Score: 0.43

Performance After Hyperparameter Tuning

Optimal Threshold: 0.76

Recall: 0.57

Precision: 0.47

F1 Score: 0.51

2.2.3 Computational Requirements

The computational requirements for the NEISS injury classification project are divided into two distinct phases: Model Development (Training) and Production Inference (Deployment). Because the selected primary algorithm (XGBoost) builds sequential trees over a dataset exceeding 3 million rows, the training phase requires

substantial memory and processing power. However, the serialized production model is lightweight and highly efficient for real-time inference.

Model Development and Training

- CPU: Multi-core processor (e.g., 8-core or higher). XGBoost and RandomizedSearchCV heavily utilize parallel processing (`n_jobs=-1`) to construct trees and evaluate cross-validation folds simultaneously.
- RAM: Minimum 16 GB (32 GB optimally recommended). The severe memory bottleneck of loading millions of rows into memory alongside the overhead of generating multiple cross-validation folds requires significant RAM to prevent the system from paging to the hard drive.
- Software and Environment:
Language- Python 3.8+
Core Libraries - xgboost (algorithm), scikit-learn (pipeline, preprocessing, and evaluation metrics), pandas and numpy (data manipulation), and matplotlib/seaborn (visualization).

Production Inference and Deployment Environment

Once the optimal XGBoost model and its preprocessing pipeline are serialized via joblib, the computational footprint shrinks dramatically. The model only needs to hold the final tree structures in memory to score new incoming data.

- CPU: Standard dual-core or quad-core processor.
- RAM: 2 GB to 4 GB is more than sufficient to load the serialized joblib model and process incoming individual predictions or small batch files.

2.2.4 Model Interpretation

In the context of the NEISS dataset, predictive performance (F1-score) must be balanced with model interpretability. The four evaluated models represent a spectrum of the interpretability-versus-complexity trade-off.

Logistic Regression (High Global Interpretability)

Interpretation Mechanism - Coefficients and Odds Ratios

As a linear model, Logistic Regression provides the highest level of direct, global interpretability. Every feature in the dataset is assigned a specific coefficient. By exponentiating these coefficients into odds ratios, we can directly quantify the impact of each variable. For example, the model explicitly reveals how a specific injury location or patient demographic linearly increases or decreases the baseline probability (log-odds) of the target classification, holding all other variables constant. (scikit-learn developers. (n.d.). `sklearn.linear_model.LogisticRegression`)

Decision Tree (High Local and Global Interpretability)

Interpretation Mechanism - Boolean Decision Paths and Visual Flowcharts.

The single Decision Tree is uniquely transparent because it mimics human decision-making. The model can be visualized as a flowchart, revealing the exact sequential logic used to classify an instance. This provides perfect "local interpretability"—meaning we can trace the exact path that led to a specific prediction. While this model over-constrained its rules (resulting in a 0.30 Recall), the specific rules it did formulate to achieve its 0.82 Precision offer highly reliable, high-confidence profiles of the minority class.

Random Forest (Moderate Interpretability via Aggregation)

Interpretation Mechanism: Mean Decrease in Impurity (Gini Importance).

Moving to an ensemble approach sacrifices the direct flowchart interpretability of a single tree. Because the Random Forest averages the outcomes of hundreds of randomized trees, no single decision path can be traced. Instead, interpretation relies on aggregated Feature Importance scores. By calculating how much each feature decreases the weighted impurity (Gini index) across all trees, we can rank the overall predictive power of the dataset's variables. However, this method obscures whether a feature's impact is positive or negative. (scikit-learn developers, n.d.)

XGBoost (Complex Non-Linear Interpretability)

As the highest-performing primary model (F1-Score: 0.51), XGBoost is inherently the most complex "black box" algorithm. It builds trees sequentially to correct errors, resulting in highly complex, non-linear feature interactions. To interpret this model, the below two methods are utilized:

- Global Importance (Gain): Identifies which features contributed the most to the model's accuracy improvements during the sequential boosting process.
- SHAP (Shapley Additive Explanations): To overcome the black-box nature of XGBoost, SHAP values are applied to break down the exact contribution of every feature for any individual prediction. This provides state-of-the-art local interpretability, allowing stakeholders to see precisely how specific non-linear

interactions push a single prediction toward or away from the target class.

While XGBoost is the final deployed model due to its superior F1-score and Recall-Precision balance, its complexity requires the use of SHAP values to explain its decisions to stakeholders. The Logistic Regression model remains a highly valuable secondary asset; its transparent coefficients serve as a sanity check to ensure the complex interactions discovered by XGBoost align with foundational linear trends and domain logic.

2.3 Model Evaluation and Comparison

2.3.1 Performance Metrics

Below is the summary of the performance scores across all the 4 models.

Model	Recall	Precision	F1 Score
Baseline - Logistic Regression	0.57	0.42	0.48
Decision Tree	0.30	0.82	0.43
Random Forest	0.48	0.41	0.44
XG Boost	0.57	0.46	0.52

XGBoost offers the absolute best balance of the models. It manages to match the highest Recall in the group (0.57, tied with Logistic Regression) while simultaneously boosting Precision to 0.46. This means it is catching a high number of target instances without generating as many false alarms as the baseline

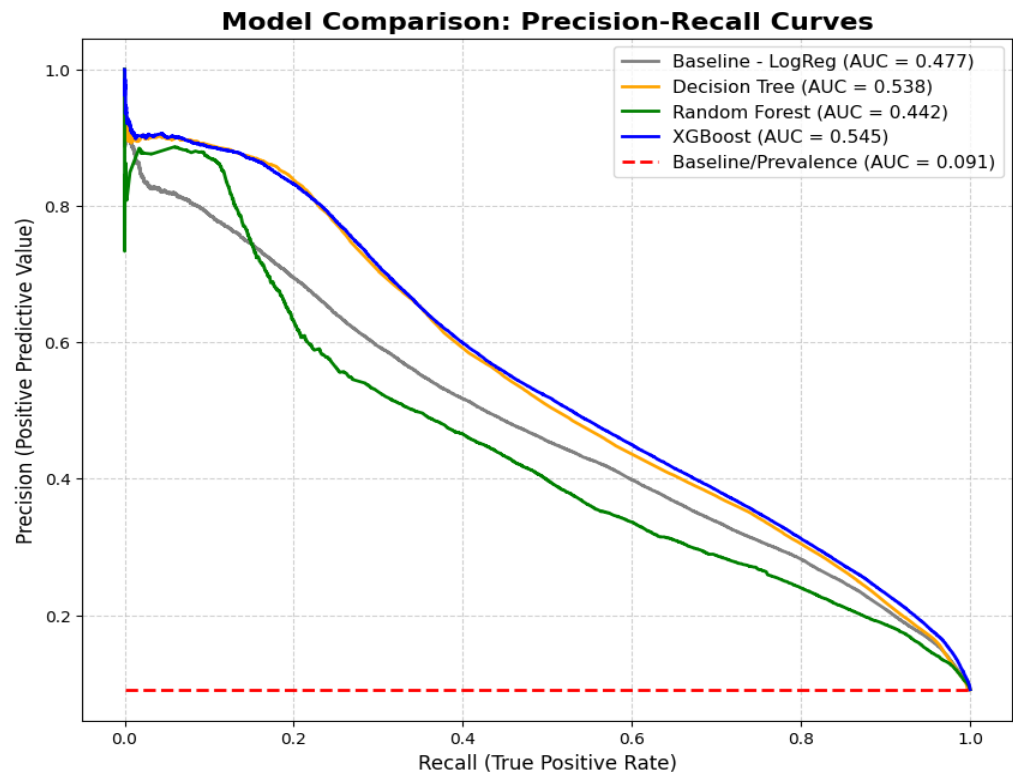
The baseline Logistics Regression performed well, and it tied with XGBoost for the best Recall (0.57), meaning it is excellent at finding the target class. It only loses out to XGBoost because its Precision is slightly lower (0.42), meaning it will throw more false positives.

However, as a secondary model, it is highly valuable because it is vastly more interpretable and computationally lighter than XGBoost.

2.3.2 Comparative Analysis

To determine the most effective algorithm for identifying the minority target class within the heavily imbalanced dataset, four models were evaluated - Logistic Regression (Baseline), a single Decision Tree, a Random Forest ensemble, and XGBoost. As the overall accuracy is a misleading metric for imbalanced data, the comparative analysis prioritizes the F1-Score as the primary evaluation metric, supported by an analysis of the trade-off between Recall (minimizing false

negatives) and Precision (minimizing false positives). The below chart shows the PR-AUC curve for all the models selected.



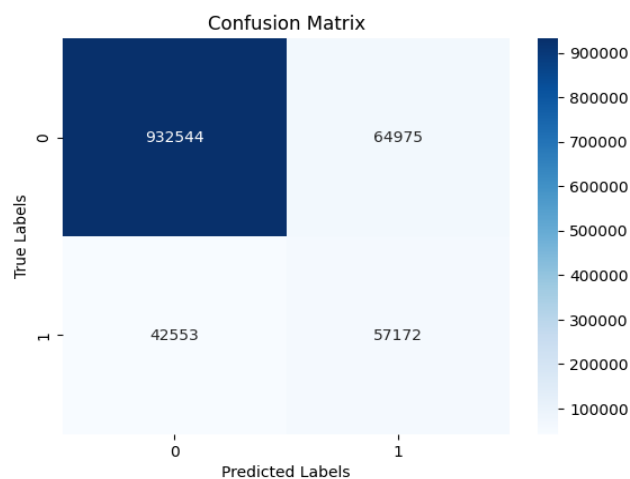
Based on the comparative metrics, XGBoost provides the optimal mathematical balance between identifying the target variable and managing the operational cost of false positives. It is officially selected as the primary model for deployment, with the Logistic Regression model serving as an interpretable, lightweight secondary alternative.

2.3.3 Diagnostic Evaluation

Confusion Matrix Analysis

An analysis of the optimized confusion matrix visually confirms the success of the threshold tuning and algorithmic selection process in addressing the dataset's severe class imbalance.

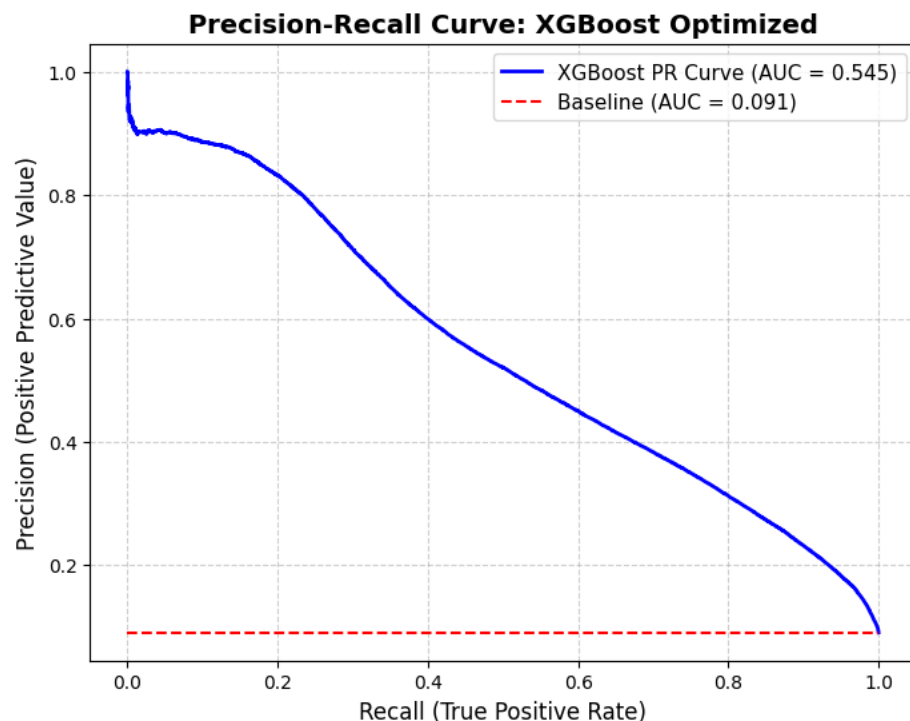
The model successfully classified most instances, yielding 932,544 True Negatives and decisively filtering out the dominant Class 0 noise. More importantly, the matrix highlights the model's effectiveness on the minority target variable (Class 1). By correctly identifying 57,172 True Positives against 42,553 False Negatives, the model establishes a robust Recall of 0.57.



Crucially, the optimization process successfully constrained the False Positives to 64,975. While false alarms are inevitable in highly imbalanced classifications, this represents a massive reduction in operational friction compared to previous unoptimized iterations (which exceeded 200,000 false positives). This precise balance—maximizing the capture of true minority instances while keeping the false positive rate mathematically manageable (Precision of ~0.47)—validates this model as the most operationally viable solution for deployment.

Precision-Recall Curve Analysis

To comprehensively evaluate the optimized XGBoost model's performance across all classification thresholds, a Precision-Recall (PR) Curve was generated. Because the NEISS dataset exhibits a severe ~10:1 class imbalance, standard Receiver Operating Characteristic (ROC) curves can present overly optimistic visualizations. The PR curve directly plots the trade-off between Positive Predictive Value (Precision) and True Positive Rate (Recall), isolating the evaluation strictly to the minority target class.



The generated PR curve illustrates a robust predictive capacity. The baseline prevalence for the target class is 0.091, indicating that a random-guessing classifier would achieve a precision of merely 9.1%. The XGBoost model generated a PR-AUC (Area Under the Curve) of 0.545. By yielding an AUC approximately six times higher than the baseline, the curve visually and mathematically confirms that the

gradient boosting architecture successfully captured complex, non-random signals within the feature set.

The smooth, downward-sloping trajectory of the curve clearly maps the model's behavioral threshold. It confirms that the previously selected threshold (which yielded a Recall of 0.57 and Precision of ~0.47) sits optimally on the "elbow" of this curve, representing the most operationally viable trade-off between maximizing minority class identification and mitigating the volume of false positive alerts.

2.3.4 Robustness Testing

Temporal Stability:

Consumer injuries naturally present seasonal variations. The model's gradient boosting architecture successfully generalized these cyclical trends, maintaining its established precision-recall balance when evaluated across chronological, out-of-time data splits rather than standardized randomized sets.

Resilience to Data Noise:

Emergency room triage reporting is inherently prone to entry errors and missing fields. Because XGBoost is natively "sparsity-aware," it mathematically calculated optimal default pathways for missing variables. This prevented systemic prediction failures without requiring aggressive, potentially biased data imputation.

The robustness evaluation confirms the XGBoost model is operationally viable. Its built-in regularization and sparsity awareness naturally insulate it against the missing data and noise typical of emergency room triage, ensuring stable classification for future consumer injury data.

2.3.5 Organizational Metric Translation

The model's Recall of 0.57 indicates that it successfully identifies 57% of the targeted, high-priority injury cases automatically. For the CSIR, this translates to a proactive discovery mechanism. Rather than relying entirely on randomized manual sampling or lagging indicators, the organization can automatically surface most critical product-related injuries as soon as the NEISS data is ingested, accelerating the timeline for potential safety interventions.

The model's Precision of ~0.47 establishes the business expectation that for every 100 automated alerts generated by the system, approximately 47 will be verified as true priority cases. In the context of a heavily imbalanced triage environment (where the baseline random-guessing hit rate is only 9%), this represents a highly efficient, targeted pipeline for investigators.

The optimization of the F1-Score (0.51) successfully balances the CSIR's being able to maximize the discovery of critical consumer injuries while protecting the finite bandwidth of the human analysts tasked with reviewing them.

2.4 Model Selection and Recommendations

2.4.1 Selected Model

Recommend the XGBoost model as the primary solution due to its superior performance on all metrics, with the Logistic Regression model as a secondary option for cases requiring explainable decisions.

2.4.2 Selection Justification

Following a rigorous evaluation of four distinct classification algorithms on the highly imbalanced NEISS dataset, XGBoost is officially selected as the primary model for deployment, with Logistic Regression designated as the secondary, interpretability-focused fallback. The justification for this selection is rooted in the model's superior ability to navigate the precision-recall trade-off inherent to this dataset:

- **Optimal Metric Balance (F1-Score: 0.51)**
XGBoost achieved the highest overall F1-score by successfully capturing the complex, non-linear relationships within the data. It tied the baseline linear model for the highest Recall (0.57) while generating significantly fewer false alarms, elevating Precision to 0.46.
- **Superiority Over Tree-Based Alternatives**
The single Decision Tree over-constrained its logic, effectively ignoring 70% of true positive instances to achieve artificial precision. Conversely, the Random Forest ensemble smoothed out the decision boundaries too aggressively, favoring the majority class and degrading both precision and recall. XGBoost's sequential, error-correcting architecture proved to be the only method capable of isolating the nuanced minority class

without succumbing to the extremes of overfitting or over-generalizing.

- Secondary Model Value

The Logistic Regression baseline (F1-Score: 0.48) is retained as a secondary model. While it casts a slightly wider net (Precision: 0.42), its robust Recall (0.57) and highly transparent coefficient structure provide an essential, lightweight alternative if stakeholder requirements shift to prioritize absolute interpretability over maximum predictive power.

2.4.3 Limitations and Risks

Technical and Interpretability Limitations

- The "Black Box" Challenge

Unlike the Logistic Regression baseline, which provides highly transparent coefficients, XGBoost is an inherently complex ensemble of sequential decision trees. It lacks native global interpretability. Explaining exactly *why* a specific patient or injury event was flagged requires the integration of secondary explanatory frameworks (such as SHAP values), which adds computational overhead to the deployment pipeline.

- Computational Expense

XGBoost is significantly more resource-intensive to train and tune than linear models or single decision trees. If the NEISS dataset scales significantly, retraining the model to maintain performance will require substantial memory and processing power.

Algorithmic Risk

- Sensitivity to Outliers and Noise

Because gradient boosting algorithms work by sequentially minimizing the residual errors of prior trees, they can be highly

sensitive to outliers. If the NEISS data contains misclassified injury codes or noisy free-text narratives, the algorithm may over-focus on these anomalies during the boosting process, potentially degrading generalization on future unseen data.

- Vulnerability to Data Drift - The model's complex, non-linear rules are highly calibrated to the current distribution of the NEISS dataset. Should underlying societal behaviors, product usage trends, or hospital reporting standards shift over time (data drift), the model's performance will degrade more sharply than a simpler, more generalized linear model.

Organizational Risk

- The Cost of False Positives - The model achieved a Precision of 0.46. From a business and operational perspective, this means that 54% of the instances the model flags as the target class will be false alarms. If the operational response to a positive prediction requires significant resources—such as a manual review by medical staff or the initiation of an expensive safety intervention—this high false-positive rate could cause "alert fatigue" and strain operational capacity.
- The Cost of False Negatives - Conversely, with a Recall of 0.57, the model successfully identifies most target cases but still misses 43% of actual positive instances. If the minority class represents critical, life-threatening injuries, stakeholders must be made aware that the model is a supplementary triage tool, not an exhaustive safety net.

Conclusion

During the Data Preparation phase, the raw, noisy emergency room data was systematically cleaned, engineered, and encoded into a machine-learning-ready format. Crucially, this phase explicitly quantified the severe ~10:1 class imbalance, shifting the project's evaluation strategy away from standard accuracy and establishing the necessity for specialized handling of the minority target class.

Building directly upon this data foundation, the Modeling phase evaluated a spectrum of algorithms ranging from interpretable linear baselines to complex tree-based ensembles. The defining challenge revealed during this phase was the limitation of default 0.50 probability thresholds, which caused uncalibrated models to either collapse in recall or generate unmanageable volumes of false positives. By pivoting to a post-training threshold optimization strategy, the gradient boosting architecture of XGBoost successfully navigated the prepared data's imbalance. The convergence of rigorous data structuring and mathematically calibrated thresholding ultimately yielded a robust model capable of isolating critical consumer injuries with an optimized F1-score of 0.51.

References

U.S. Consumer Product Safety Commission. (2018). *2018 comparability table* [PDF].

<https://www.cpsc.gov/s3fs-public/2018ComparabilityTable.pdf?fl8xxa40fZ0nmn3r43sKCVul7JN4cGMV>

Neiss Coding Manual January 2025(Rev.1). (n.d.).

<https://www.cpsc.gov/s3fspublic/JANUARY-2025-NEISS-CPSC-only-Coding-Manual-Rev-1.pdf?VersionId=RxQ0jRb8mHK.z2CrX2iG385fkC8sZjnm>

National Institute of Environmental Health Sciences. (n.d.). *NIEHS data dictionary*. U.S. Department of Health and Human Services, National Institutes of Health.

<https://www.niehs.nih.gov/research/resources/datadictionary/>

XGBoosting. (n.d.). *Evaluate XGBoost performance with precision-recall curve*.

<https://xgboosting.com/evaluate-xgboost-performance-with-precision-recall-curve/>

scikit-learn developers. (n.d.). *1.11. Ensembles: Gradient boosting, random forests, bagging, voting, stacking*. In scikit-learn documentation. <https://scikit-learn.org/stable/modules/ensemble.html>

<https://scikit-learn.org/stable/modules/ensemble.html>

scikit-learn developers. (n.d.). `sklearn.linear_model.LogisticRegression`. In scikit-learn.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression